



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2283 — Diseño y Análisis de Algoritmos 2025-2

Interrogación 1

27 de septiembre de 2025

Evaluación: Cada pregunta tiene 6 puntos (+1 punto base). La nota es el promedio de las 3 preguntas.

Forma de Entrega: Además de entregar las hojas con sus respuestas, debe subir una versión digital a Canvas. Al final de la prueba, tiene 10 minutos para subir la imagen de la prueba, en formato vertical, **cada pregunta por separado**. Es su responsabilidad asegurarse de que todas sus respuestas se suban correctamente al sistema.

Dispositivo: Para digitalizar la prueba se permite el uso de UN dispositivo (teléfono, tablet) el cual debe estar guardado y en silencio durante el desarrollo.

Torpedo: Se permite una hoja tamaño oficio escrita a mano por usted, usando ambos lados. También debe subirse una versión digital a Canvas.

Política de Salidas: Si necesita salir de la sala al baño, debe dejar el celular adelante en la mesa con los ayudantes.

Se sugiere leer todas las preguntas antes de iniciar las respuestas.

Tiempo: 2 horas.

1. Dividir y Conquistar, Teorema Maestro

Sea

$$A = \begin{pmatrix} a_1 & a_2 & \dots & a_n \\ b_1 & b_2 & \dots & b_n \end{pmatrix}$$

una matriz de dimensión $2 \times n$, representada por filas usando la secuencia (o arreglo) $(a_1, a_2, \dots, a_n, b_1, b_2, \dots, b_n)$. El objetivo de esta pregunta es diseñar un algoritmo de tipo *Dividir y Conquistar* para obtener la matriz traspuesta

$$A^T = \begin{pmatrix} a_1 & b_1 \\ a_2 & b_2 \\ \vdots & \vdots \\ a_n & b_n \end{pmatrix},$$

representada por filas mediante la secuencia $(a_1, b_1, a_2, b_2, \dots, a_n, b_n)$. Por ejemplo, la matriz

$$A = \begin{pmatrix} 5 & -7 & 3 & 1 \\ 9 & 12 & -3 & 6 \end{pmatrix}$$

se representa por filas con la secuencia $(5, -7, 3, 1, 9, 12, -3, 6)$. La representación por filas de A^T en este caso debe ser la secuencia $(5, 9, -7, 12, 3, -3, 1, 6)$.

- a) **[3.0 puntos]** Diseñe (y escriba) un algoritmo de tipo **Dividir y Conquistar** que resuelva este problema de manera in-place (es decir, usando una cantidad de memoria adicional $\Theta(1)$ por sobre el espacio usado por la matriz de entrada, y exceptuando el espacio requerido por la recursión). Una solución que usa un arreglo adicional es trivial y no se va a considerar. Puede usar pseudocódigo o cualquier lenguaje de programación. Su solución debe ser eficiente.

Solución: Haremos el diseño del dividir y conquistar usando inducción, lo cual no sólo permite definir el algoritmo, sino que también permite demostrar su correctitud. Llamaremos `trasponer`(A, n) al algoritmo que calcula A^T en el formato solicitado. Asumiremos que n es una potencia de 2 para simplificar el diseño.

- **Caso Base:** Para $n = 1$, tenemos la matriz $A = (a_1, b_1)$, cuya traspuesta es `trasponer`($A, 1$) = (a_1, b_1) .
- **Hipótesis Inductiva:** para todo $n' \in \{1, 2, \dots, n-1\}$, si $A = (a_1, \dots, a_{n'}, b_1, \dots, b_{n'})$, asumimos que `trasponer`(A, n') produce el resultado correcto $(a_1, b_1, a_2, b_2, \dots, a_{n-1}, b_{n-1})$.
- **Paso Inductivo:** Para $n \geq 2$, tenemos $A[1..2n] = (a_1, \dots, a_n, b_1, \dots, b_n)$. Para poder usar la hipótesis inductiva, debemos reducir el tamaño de A . Por ejemplo, si dividimos A en dos mitades A_1 y A_2 , sobre ellas se cumpliría la hipótesis inductiva (al tener un tamaño menor a n) siempre y cuando A_1 y A_2 contengan en su primera mitad elementos del tipo a_i y en su segunda mitad elementos del tipo b_i . Para lograr eso, definimos:

- $A_1[1..n] = (a_1, \dots, a_{\frac{n}{2}}, b_1, \dots, b_{\frac{n}{2}})$, y
- $A_2[1..n] = (a_{\frac{n}{2}+1}, \dots, a_n, b_{\frac{n}{2}+1}, \dots, b_n)$.

Ahora, dado que A_1 y A_2 tienen tamaño $n/2 < n$ y tienen el formato asumido, entonces:

- $A'_1[1..n] = \text{trasponer}(A_1, n/2)$, lo cual por hipótesis inductiva produce $A'_1 = (a_1, b_1, \dots, a_{\frac{n}{2}}, b_{\frac{n}{2}})$.
- $A'_2[1..n] = \text{trasponer}(A_2, n/2)$, lo cual por hipótesis inductiva produce $A'_2 = (a_{\frac{n}{2}+1}, b_{\frac{n}{2}+1}, \dots, a_n, b_n)$.

Note que sólo resta concatenar esas matrices obtenidas para producir el resultado esperado:

$$A^T = A'_1 \cdot A'_2 = (a_1, b_1, \dots, a_{\frac{n}{2}}, b_{\frac{n}{2}}, a_{\frac{n}{2}+1}, b_{\frac{n}{2}+1}, \dots, a_n, b_n).$$

Eso define nuestro algoritmo, el cual es implementado de la siguiente manera para cumplir con el hecho de que debe ser in-place:

Algoritmo 1: $\text{trasponer}(A, l, r)$

```

1  $n \leftarrow (r - l + 1)/2$ 
2 if  $n = 1$  then
3   | return
4 else
5   |  $m \leftarrow n/2$ 
6   | for  $i = 1$  to  $m$  do
7     | Intercambiar  $A[l + m + i - 1]$  (es decir,  $a_i$ ) con  $A[l + n + i - 1]$  (es decir,  $b_i$ ).
8   | end
9   |  $\text{trasponer}(A, l, l + 2m - 1)$ 
10  |  $\text{trasponer}(A, l + 2m, r)$ 
11 end
```

Inicialmente el algoritmo se invoca con $\text{trasponer}(A, 1, 2n)$. Note que el algoritmo trabaja sobre el mismo arreglo A , necesitando espacio adicional $\Theta(1)$ para hacer el trabajo (más el espacio de la recursión que es $O(\log n)$ en este caso).

Rúbrica: Se evalúa la técnica Dividir y Conquistar, por lo que los algoritmos deben tener las 3 partes que indica la técnica (en este caso, caso base, parte dividir, parte conquistar). Aunque hice la demostración por inducción para demostrar la correctitud del algoritmo y facilitar su diseño, tener en cuenta que esa demostración se evalúa en el punto b) de la pregunta. Por lo tanto, la pauta de evaluación es la siguiente:

- (I) Caso base: **10 %** del puntaje.
- (II) Dividir: Debía dividir en 2 mitades, según lo indica la ecuación de recurrencia, y reorganizar los elementos para que cumplan con el formato indicado. **70 %** del puntaje

(III) Conquistar: 2 llamadas recursivas, según lo indica la ecuación de recurrencia. **20 %** del puntaje.

- b) **[1.5 puntos]** Demuestre formalmente que su algoritmo es correcto usando inducción (esto puede ser parte del punto a) si diseña su algoritmo usando inducción, tal como se estudió en clases).

Solución: ya resuelto por inducción en la parte a).

Rúbrica:

(I) Caso base: **20 %** del puntaje.

(II) Hipótesis inductiva: **20 %** del puntaje

(III) Paso inductivo: **60 %** del puntaje.

- c) **[1.5 puntos]** Plantee la ecuación de recurrencia asociada al tiempo de ejecución de su algoritmo. Acote dicha ecuación usando notación asintótica, usando la herramienta que considere necesaria.

Solución: El tiempo de ejecución del algoritmo propuesto está dado por la siguiente ecuación de recurrencia, la cual mide la cantidad de intercambios de elementos de la matriz realizados (se podría haber usado alguna otra medida de tiempo, valorarlas):

$$T(n) = \begin{cases} 0, & n = 1 \\ 2T(\frac{n}{2}) + \frac{n}{4}, & n > 1. \end{cases}$$

(Punto (I) de la rúbrica) Usando el Teorema Maestro, tenemos que $a = 2$, $b = 2$, $\lg_b a = \lg_2 2 = 1$, y $f(n) = n/4$. Aquí se cumple la condición 2) del Teorema, ya que $f(n) \in \Theta(n^{\lg_2 2}) = \Theta(n)$, **(Punto (II) de la rúbrica)** por lo que $T(n) \in \Theta(n \lg_2 n)$.

Rúbrica:

(I) Indica el caso del Teorema que aplica: **50 %** del puntaje.

(II) Resuelve correctamente el caso: **50 %** del puntaje.

2. Cotas inferiores y análisis de caso promedio

1. [3.0 puntos] Considere el algoritmo INSERTIONSORT para ordenar un arreglo $A[1..n]$:

Algoritmo 2: INSERTIONSORT(A, n)

```
1  $i \leftarrow 2$ 
2 while  $i \leq n$  do
3    $j \leftarrow i$ 
4   while  $j > 1$  and  $A[j] < A[j - 1]$  do
5     Intercambiar  $A[j]$  y  $A[j - 1]$ 
6      $j \leftarrow j - 1$ 
7   end
8    $i \leftarrow i + 1$ 
9 end
```

Obtenga la cantidad promedio de comparaciones $A[j] < A[j - 1]$ que se hacen en la línea 4 del algoritmo al invocarlo con INSERTIONSORT(A, n).

Solución: Para $i = 2, \dots, n$, hay que considerar la inserción de $A[i]$ en el segmento ordenado $A[1..i - 1]$, teniendo en cuenta que podría ser insertado en cualquiera de las i posiciones dentro del segmento ordenado. Asumiremos que cada una de esas posiciones ocurre con la misma probabilidad $1/i$ (Punto (I) de la rúbrica). Así, tenemos que el costo promedio para cada elemento $A[i]$ es $\frac{1}{i} \sum_{j=1}^i j$ (Punto (II) de la rúbrica). Considerando todos los elementos del arreglo, la cantidad promedio de comparaciones realizadas es (Puntos (III) y (IV) de la rúbrica):

$$\sum_{i=1}^{n-1} \left(\frac{1}{i} \sum_{j=1}^i j \right) = \sum_{i=1}^{n-1} \frac{1}{i} \frac{i(i+1)}{2} = \sum_{i=1}^{n-1} \frac{(i+1)}{2} \approx \frac{n^2}{4}.$$

Rúbrica:

- (I) Asume igual probabilidad de casos: **5 %** del puntaje.
- (II) Modela todos los casos posibles para $A[i]$: **50 %** del puntaje.
- (III) Plantea la suma de casos: **20 %** del puntaje.
- (IV) Obtiene el resultado correcto para la suma: **25 %** del puntaje.

El análisis también puede hacerse mediante la media de variables aleatorias adecuadamente definidas.

2. [3.0 puntos] Bob piensa un número real $x \in [1, n]$, en donde $n \geq 1$ es un número entero. Alice debe adivinar el número pensado por Bob haciendo preguntas de comparación de la forma “¿ $x \leq t$?”, o “¿ $x = t$?”, para cualquier $t \in \mathbb{R}$ elegido por ella, y las respuestas de Bob antes esas preguntas son “sí/no”. Dado que el número real x es

arbitrario, podría ser muy difícil adivinarlo y el juego dejaría de tener sentido. Para solucionar esto, Alice y Bob se ponen de acuerdo en que en lugar de adivinar x , Alice debe adivinar el intervalo más pequeño posible que contiene a x , usando el siguiente protocolo:

- Si x es un entero $k \in \{1, \dots, n\}$ (es decir, x es uno de los n números enteros contenidos en $[1, n]$), la respuesta correcta es el intervalo puntual $[x, x]$.
- Si x no es un entero, la respuesta correcta es el intervalo unitario abierto $(i, i + 1)$, tal que $x \in (i, i + 1)$ para algún $i \in \{1, 2, \dots, n - 1\}$.

Demuestre formalmente que cualquier estrategia que use comparaciones para adivinar el intervalo que contiene a x necesita al menos $\lceil \log_2 n \rceil + 1$ comparaciones en el peor caso.

Solución: (Punto (I) de la rúbrica) Cualquier estrategia que plantee Alice puede ser representada como un árbol de decisión tal que:

- Los nodos internos del árbol corresponden a las comparaciones hechas por Alice al intentar adivinar el número x pensado por Bob. Cada nodo contiene una única comparación del tipo “ $x \leq t$ ” o “ $x = t$ ”, para algún t arbitrario decidido por Alice.
- Las ramas del árbol corresponden a las decisiones tomadas en base a las comparaciones hechas por el algoritmo.
- Estando en un nodo del árbol de decisión, el estado del algoritmo hasta ese punto de la ejecución se mantiene de forma implícita por el camino que se ha seguido.
- Los nodos externos del árbol corresponden a las distintas salidas del algoritmo, es decir, son del tipo $[i, i]$ o $(i, i + 1)$, de acuerdo a los tipos de respuestas que hay que dar. El árbol debe una cantidad de nodos externos $\geq$ a la cantidad de posibles salidas distintas.

(Punto (II) de la rúbrica) Respecto a este último punto, la clave para calcular la cota inferior está en determinar la cantidad de salidas distintas que cualquier algoritmo que resuelva este problema debería ser capaz de producir. Note que la cantidad de salidas en este caso debería ser $L \geq n + n - 1 = 2n - 1$, que es la cantidad de distintos tipos de intervalos que podemos necesitar responder dentro del intervalo original $[1..n]$. (Punto (III) de la rúbrica) Como consecuencia, cualquiera de dichos árboles de decisión tiene altura:

$$h \geq \lceil \log_2 (2n - 1) \rceil.$$

(Punto (IV) de la rúbrica) Manipulando esta expresión, tenemos que:

$$\begin{aligned} h &\geq \lceil \log_2 (2n - 1) \rceil \\ &\approx \lceil \log_2 (2n) \rceil \\ &= \lceil \log_2 (n) \rceil + 1. \end{aligned}$$

Dado que la altura del árbol de decisión corresponde a la cantidad de comparaciones realizadas por un algoritmo en el peor caso, tenemos que $\lceil \log_2(n) \rceil + 1$ es una cota inferior para el problema planteado.

Rúbrica:

- (I) Explica a grandes rasgos cómo sería cualquier árbol de decisión para resolver este problema (nodos internos, ramas del árbol, nodos externos): **10 %** del puntaje.
- (II) Plantea cuántos nodos externos debería tener cualquiera de estos árboles de decisión para resolver el problema correctamente: **60 %** del puntaje.
- (III) Acota inferiormente la altura del árbol de acuerdo al valor obtenido en el punto anterior (si el valor del punto anterior es inválido pero acota correctamente con ese valor, asignar todo el puntaje): **10 %** del puntaje.
- (IV) Manipula la expresión anterior para obtener el resultado final: **20 %** del puntaje.

3. Programación Dinámica

Considere el problema de calcular la distancia de edición entre dos strings, estudiado en clases. Dados dos strings $w_1[1..n], w_2[1..m] \in \Sigma^*$, su distancia de edición se define como el mínimo número de operaciones de edición requeridas para transformar w_1 en w_2 . Sea $w[1..n] \in \Sigma^*$ con $w = a_1 \cdots a_n$ y $n \geq 0$, las operaciones de edición estudiadas en clases son las siguientes:

- **cambiar**(w, i, b) = $a_1 \cdots a_{i-1}ba_{i+1} \cdots a_n$, para $1 \leq i \leq n$, $b \in \Sigma$.
- **agregar**(w, i, b) = $a_1 \cdots a_i ba_{i+1} \cdots a_n$, para $0 \leq i \leq n$, $b \in \Sigma$.
- **eliminar**(w, i) = $a_1 \cdots a_{i-1}a_{i+1} \cdots a_n$, para $1 \leq i \leq n$.

Sin embargo, a este conjunto de operaciones vistas en clases se le ha añadido la siguiente operación:

- **transponer**(w, i) = $a_1 \cdots a_{i+1}a_i \cdots a_n$, para $1 \leq i < n$, $n \geq 2$.

Ejemplo: Con esta nueva definición, la distancia de edición entre $w_1[1..9] = \text{"lagoritmo"}$ y $w_2[1..9] = \text{"algoritmo"}$ es 1, dado que la operación **transponer**(lagoritmo, 1) = algoritmo. Note que con la definición vista en clases (es decir, sin la operación **transponer**) la distancia sería 2.

- a) [2.5 puntos] Escriba el algoritmo $ed(w_1[1..n], w_2[1..m])$ de **fuerza bruta recursiva**, el cual obtenga el mínimo número de operaciones para transformar w_1 en w_2 . Para mostrar su solución puede usar una ecuación de recurrencia, pseudo-código, o el lenguaje de su preferencia. Aún cuando haya maneras alternativas de resolver este problema, su solución debe ser por **fuerza bruta recursiva**, para poder aplicar programación dinámica en el siguiente punto.

Solución: La ecuación de recurrencia para el algoritmo de fuerza, es similar a la vista en clases. Entonces, definimos la distancia de edición como:

$$ed(i, j) = ed(w_1[1..i], w_2[1..j]).$$

Ahora, a lo visto en clases añadimos el caso cuando se puede transponer dos caracteres. Note que la operación **transponer** cuando $n \geq 2$ y $m \geq 2$, además de que $w_1[n] = w_2[m-1]$ y $w_1[n-1] = w_2[m]$.

$$ed(n, m) = \begin{cases} \text{máx}\{n, m\} & , n = 0 \text{ ó } m = 0; \\ \text{mín} = \begin{cases} ed(n-1, m-1) + dif(n, m) & , n > 0, m > 0; \\ ed(n, m-1) + 1 & , m > 0; \\ ed(n-1, m) + 1 & , n > 0; \\ ed(n-2, m-2) + 1 & , n \geq 2, m \geq 2, \\ & w_1[n] = w_2[m-1], w_1[n-1] = w_2[m]. \end{cases} \end{cases}$$

Rúbrica:

- (I) Casos de las 3 operaciones vistas en clases: **cambiar**, **agregar** y **eliminar**: **0.5 puntos**
- (II) Identifica que solo puede aplicar la operación **transponer** cuando $n \geq 2 \wedge m \geq 0$ y $w_1[n] = w_2[m-1] \wedge w_1[n-1] = w_2[m]$: **1.0 puntos**
- $n \geq 2 \wedge m \geq 0$: **0.5 puntos**
 - $w_1[n] = w_2[m-1] \wedge w_1[n-1] = w_2[m]$: **0.5 puntos**
- (III) Identifica que el caso recurrente: $ed(n-2, m-2)$, es decir, se entiende el proceso de transformar $w_1[1..n-2]$ en $w_2[1..m-2]$ y luego **transponer**($w_1, n-1$): **1.0 puntos**.
- b) [2.5 puntos] Diseñe un algoritmo de **programación dinámica bottom-up** (es decir, un algoritmo iterativo) que permita resolver de forma eficiente el problema. Use pseudo-código o el lenguaje de su preferencia.

Solución:

- Se utilizará una tabla $M[0..n, 0..m]$ para almacenar los subproblemas. (Punto (I) de la rúbrica)
- Llenado de la tabla (Punto (II) de la rúbrica)

Casos base:

- a) $M[0, 0] = 0$;
- b) $M[i, 0] = i$, para $i = 1, \dots, n$;
- c) $M[0, j] = j$, para $j = 1, \dots, m$.

Ahora, cada celda de la tabla M se puede llenar de la siguiente forma :

Si $i, j > 0$ y $w_1[i] \neq w_2[j-1] \vee w_1[i-1] \neq w_2[j]$

$$M[i, j] = \min \begin{cases} M[i-1, j-1] + dif(i, j) \\ M[i, j-1] + 1 \\ M[i-1, j] + 1 \end{cases}$$

Si $i, j > 1$ y $w_1[i] = w_2[j-1] \wedge w_1[i-1] = w_2[j]$

$$M[i, j] = \min \begin{cases} M[i-1, j-1] + dif(i, j) \\ M[i, j-1] + 1 \\ M[i-1, j] + 1 \\ M[i-2, j-2] + 1 \end{cases}$$

- La tabla se llena de izquierda a derecha por filas (aunque también se puede por columnas). (Punto (III) de la rúbrica)

- (Punto (IV) de la rúbrica) La solución al problema se encuentra en la casilla $M[n, m]$.
- Entonces, el algoritmo queda de la siguiente forma:

Algoritmo 3: $\text{ed}(w_1[1..n], w_2[1..m])$

```

1   $M[0..n, 0..m]$ 
2   $M[0, 0] \leftarrow 0$ 
3  for  $i \leftarrow 1$  to  $n$  do
4     $M[i, 0] \leftarrow i$ 
5  for  $j \leftarrow 1$  to  $m$  do
6     $M[0, j] \leftarrow j$ 
7  for  $i \leftarrow 1$  to  $n$  do
8    for  $j \leftarrow 1$  to  $m$  do
9      if  $i \geq 2$  and  $j \geq 2$  and  $w_1[i] = w_2[j - 1]$  and  $w_1[i - 1] = w_2[j]$  then
10          $M[i, j] = \min \begin{cases} M[i - 1, j - 1] + \text{dif}(i, j) \\ M[i, j - 1] + 1 \\ M[i - 1, j] + 1 \\ M[i - 2, j - 2] + 1 \end{cases}$ 
11       else
12          $M[i, j] = \min \begin{cases} M[i - 1, j - 1] + \text{dif}(i, j) \\ M[i, j - 1] + 1 \\ M[i - 1, j] + 1 \end{cases}$ 
13 return  $M[n, m]$ 

```

Rúbrica:

- (I) Definición de la tabla: **0.5 puntos**
 - (II) Relaciona la recurrencia de la parte a) con el llenado de la tabla: **1.0 puntos**
 - (III) Especifica orden de llenado de la tabla (si escribió código, la iteración sirve): **0.5 puntos**
 - (IV) Indica qué entrada de la tabla corresponde al resultado: **0.5 puntos**
- c) **[1.0 puntos]** Realice un análisis de tiempo de ejecución y espacio utilizado por el algoritmo del punto b), recuerde utilizar correctamente la notación asintótica.

Solución:

- El tiempo de ejecución es $\Theta(n \cdot m)$, dado que el calculo de cada una de las $(n + 1) \times (m + 1)$ casillas toma tiempo $O(1)$.
- El espacio adicional utilizado por el algoritmo es $\Theta(n \cdot m)$.

Rúbrica:

- (I) Análisis del tiempo de ejecución del algoritmo del punto b): **0.5 puntos**
- (II) Análisis del espacio utilizado por el algoritmo del punto b): **0.5 puntos**

Glosario

Teorema Maestro: Sea $f : \mathbb{N} \rightarrow \mathbb{R}_0^+$, $a, b, c \in \mathbb{R}_0^+$ tales que $a \geq 1$ y $b > 1$, y $T(n)$ una función definida por la siguiente ecuación de recurrencia:

$$T(n) = \begin{cases} c, & n = 0 \\ a \cdot T(\lfloor \frac{n}{b} \rfloor) + f(n), & n \geq 1 \end{cases}$$

Se tiene que:

1. Si $f(n) \in O(n^{\log_b(a)-\varepsilon})$ para $\varepsilon > 0$, entonces $T(n) \in \Theta(n^{\log_b(a)})$.
2. Si $f(n) \in \Theta(n^{\log_b(a)})$ para $k \geq 0$, entonces $T(n) \in \Theta(n^{\log_b(a)} \cdot \log_2(n))$.
3. Si $f(n) \in \Omega(n^{\log_b(a)+\varepsilon})$ para $\varepsilon > 0$ y f es (a, b) -regular, entonces $T(n) \in \Theta(f(n))$.

Funciones (a, b) -regulares: Sea $f : \mathbb{N} \rightarrow \mathbb{R}_0^+$ una función y $a, b \in \mathbb{R}$ constantes tales que $a \geq 1$ y $b > 1$. La función f es (a, b) -regular si existen constantes $c \in \mathbb{R}^+$ y $n_0 \in \mathbb{N}$ tales que $c < 1$ y

$$a \cdot f\left(\left\lfloor \frac{n}{b} \right\rfloor\right) \leq c \cdot f(n), \quad \forall n \geq n_0.$$

Coeficiente Binomial: Para $0 \leq k \leq n$ se tiene

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

Coeficiente Multinomial: Para $\sum_{i=1}^{\ell} v_i = m$ se tiene

$$\binom{m}{v_1, v_2, \dots, v_{\ell}} = \frac{m!}{v_1! \cdot v_2! \cdot \dots \cdot v_{\ell}!}.$$

Aproximación de Stirling para $n!$: Para n suficientemente grande, se tiene

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n.$$

Consecuencias de la Aproximación de Stirling: de la aproximación de Stirling se puede ver que:

$$\left(\frac{n}{e}\right)^n \leq n! \leq n^n,$$

y para $1 \leq v \leq m$ se tiene

$$\left(\frac{m}{v}\right)^v \leq \binom{m}{v} \leq \frac{m^v}{v!} \leq \left(\frac{e \cdot m}{v}\right)^v.$$